Cosi 231 - Spring 2025

PA3 Fine-Tuning RoBERTa for Question Answering and Decoder for Python Code Generation

Author: Danny Rechitsky

Introduction

Two tasks were completed in this project. For Task 1, a RoBERTa-based model was fine-tuned on SQuAD v2 for question-answering under several training-data and LoRA-rank configurations. Task 2 involved fine-tuning a Mellum decoder-only model for Python code generation on the HuggingFace dataset flytech/python-codes-25k. In both tasks, the goal was to determine the effects of LoRA rank and training size on model performance. The results consistently showed that higher training sizes and higher LoRA ranks produced improved evaluation metrics.

2. Methods / Preprocessing

Task 1 - RoBERTa QA

The raw dataset was loaded using the datasets library's load_dataset function. An AutoTokenizer pretrained on roberta-base was instantiated to convert raw text into the format required by RoBERTa models (for example, by adding a token at the start of each sentence or sentence pair). Text was tokenized into model inputs and sequence IDs were used to identify the start and end tokens of the context and answer in each example; identifying these spans made it possible to determine whether the answer was contained in the context and to localize it, thereby enabling the model to focus attention on the relevant context tokens when predicting answer spans. The tokenized data were split into training, validation, and test sets: SQuAD v2 provides 'train' and 'validation' splits, the original 'train' split was further divided so that 90% was used for training and 10% for validation, and the original 'validation' split was used as the test set. For experiments that used only a portion of the training data (30% and 50%), the required slice was taken from the start of the tokenized training set prior to training; the validation and test sets were left unchanged.

Task 2 - Decoder Generated Python Code

The raw dataset was loaded using the datasets library's load_dataset function. The raw datasets were split 80/20 into a test set and train set. The train set had 10% set aside for validation. The instruction/python-code pairs in the dataset were formatted for batch processing as expected by the Mellum model, including left-padding to enable proper NTP. The SFTTrainer made it possible to combine autotokenization with pre-formatting in a single call instantiating the trainer to prepare the dataset for training with the Mellum model.

3. Experiments & Training Curves

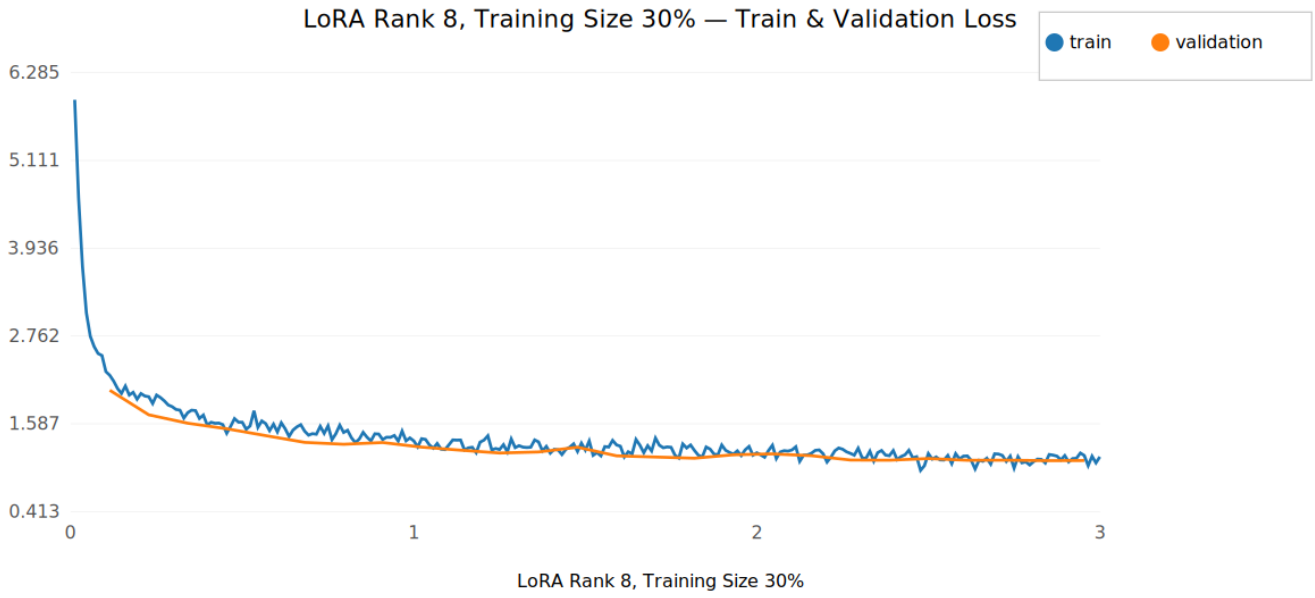
For both tasks, experiments were run under the following configurations for LoRA rank and training set size as well as a baseline pretrianed model:

- `pretrained`
- `rank=8_size=0.3`
- `rank=8_size=0.5`
- `rank=8_size=1.0`
- `rank=16_size=1.0`
- `rank=32_size=1.0`

Task 1 - RoBERTa QA

3.1 LoRA Rank 8, Training Size 30%

Loss vs Epoch

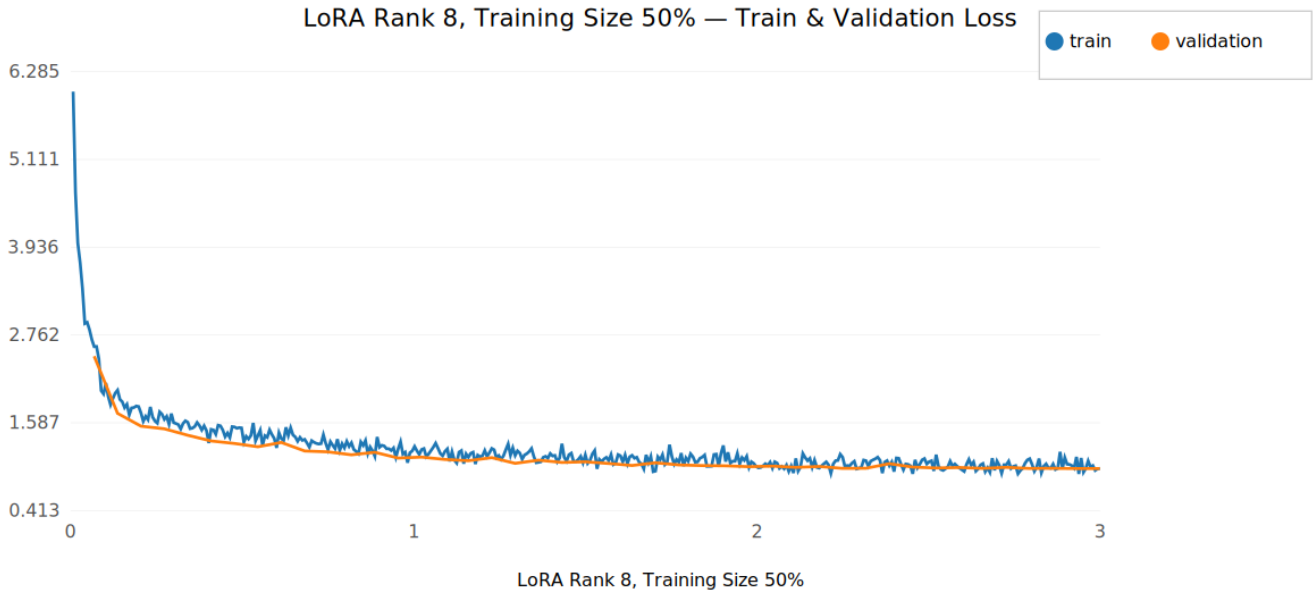


Run summary:

- Final train loss: 1.0959
- Evaluation (selected metrics):
  - Exact match: 61.4347
  - F1: 70.3291

3.2 LoRA Rank 8, Training Size 50%

Loss vs Epoch

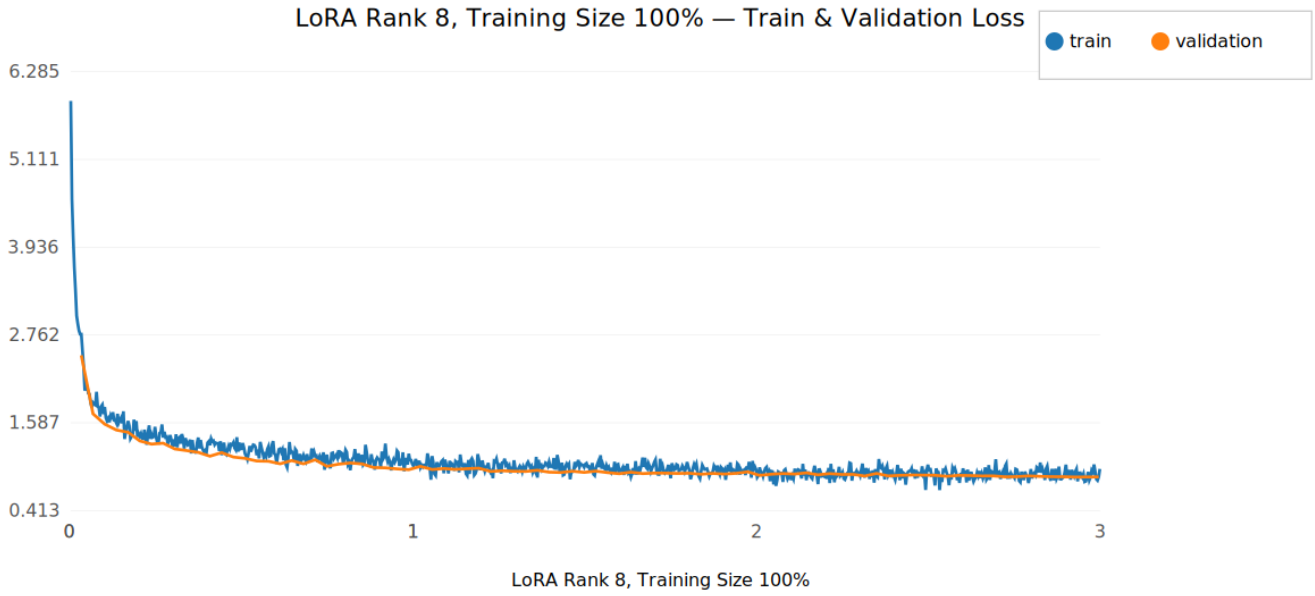


Run summary:

- Final train loss: 0.9749
- Evaluation (selected metrics):
  - Exact match: 65.0904
  - F1: 74.0327

3.3 LoRA Rank 8, Training Size 100%

Loss vs Epoch

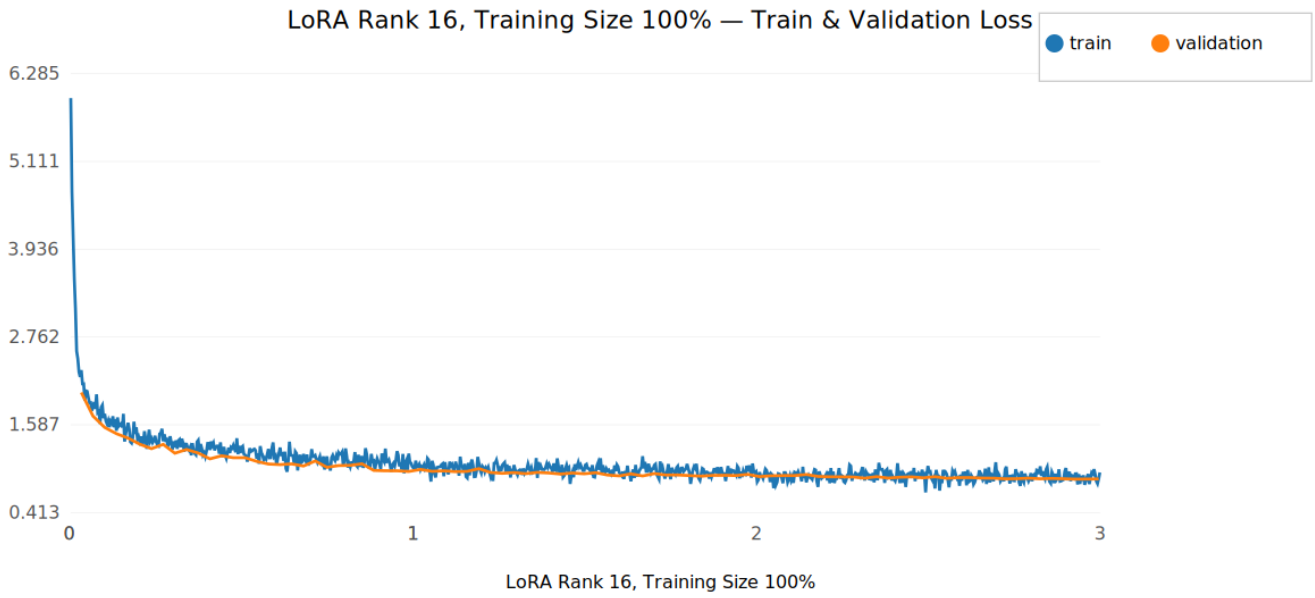


Run summary:

- Final train loss: 0.8633438944816589
- Evaluation (selected metrics):
  - Exact match: 68.0717
  - F1: 77.0880

3.4 LoRA Rank 16, Training Size 100%

Loss vs Epoch

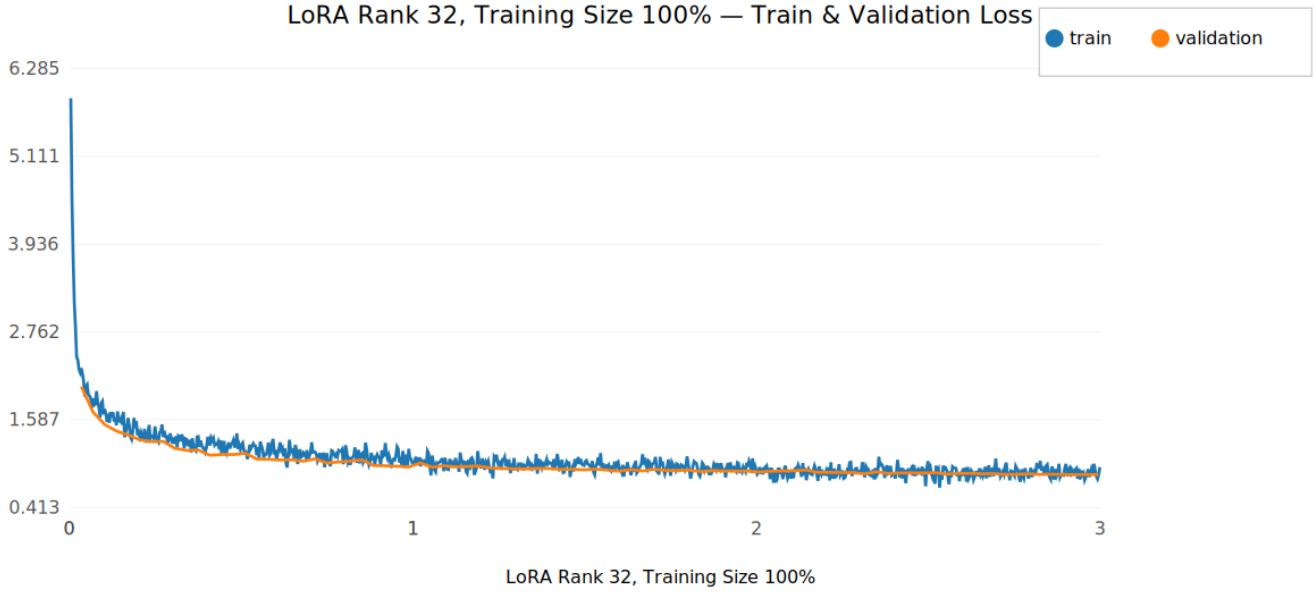


Run summary:

- Final train loss: 0.8623953461647034
- Evaluation (selected metrics):
  - Exact match: 67.8878
  - F1: 76.8095

3.5 LoRA Rank 32, Training Size 100%

Loss vs Epoch



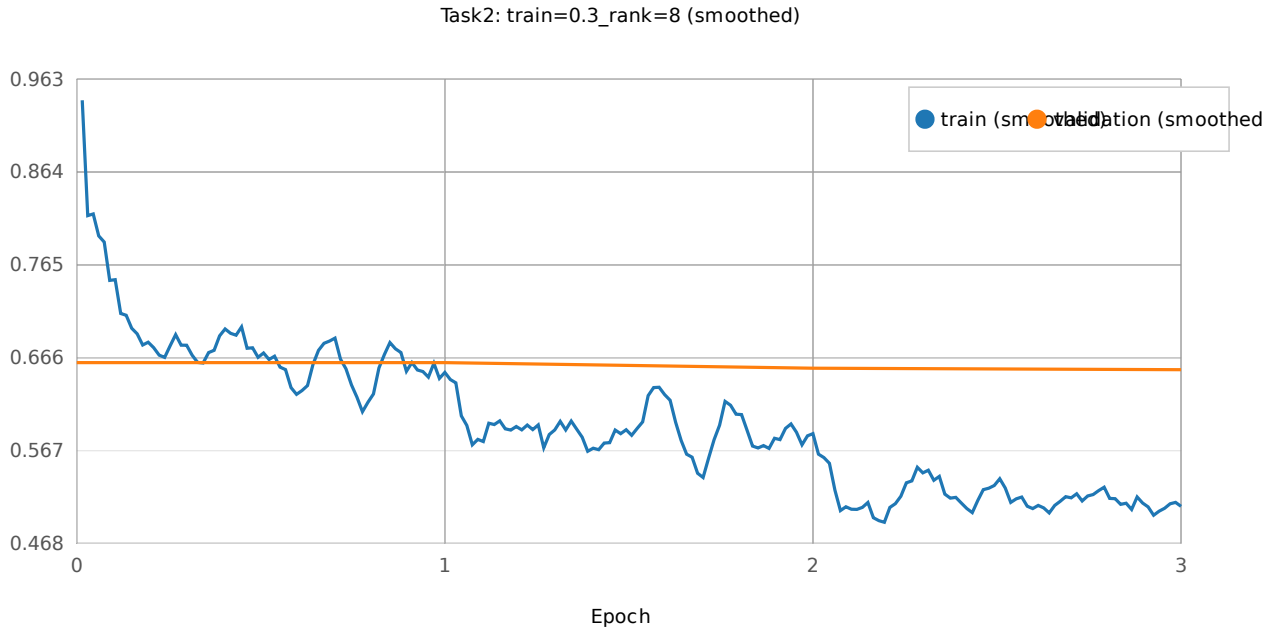
Run summary:

- Final train loss: 0.8542709946632385
- Evaluation (selected metrics):
  - Exact match: 68.0871
  - F1: 77.0862

Task 2 — Decoder Generated Python Code

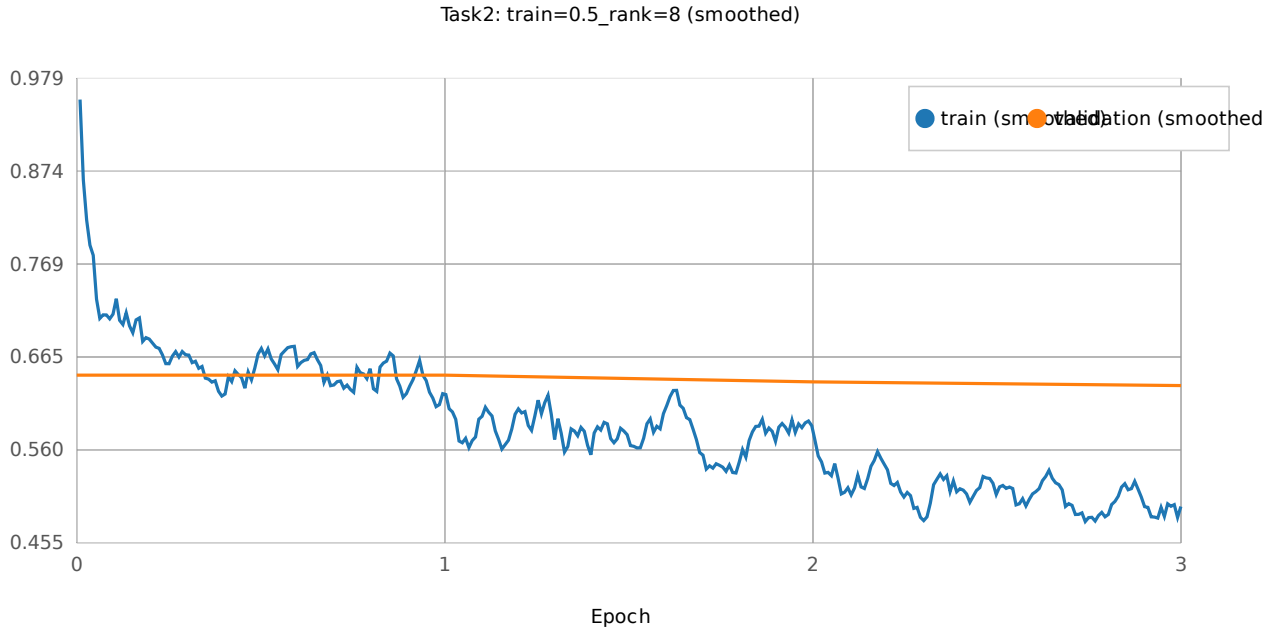
Below are per-run Task 2 training and validation loss curves. Each figure shows **training loss** (blue) and **validation loss** (orange).

- `train=0.3_rank=8`:



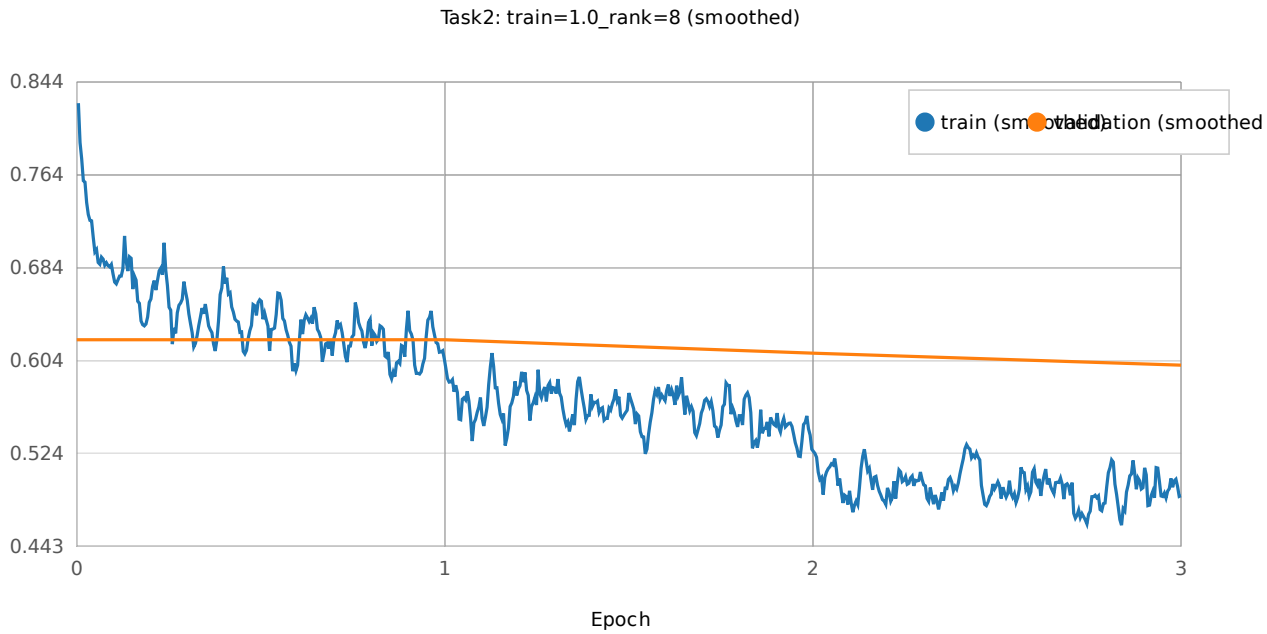
(PNG: `figures/task2_train-0p3_rank-8_train_val_smoothed.png`)

- `train=0.5_rank=8`:



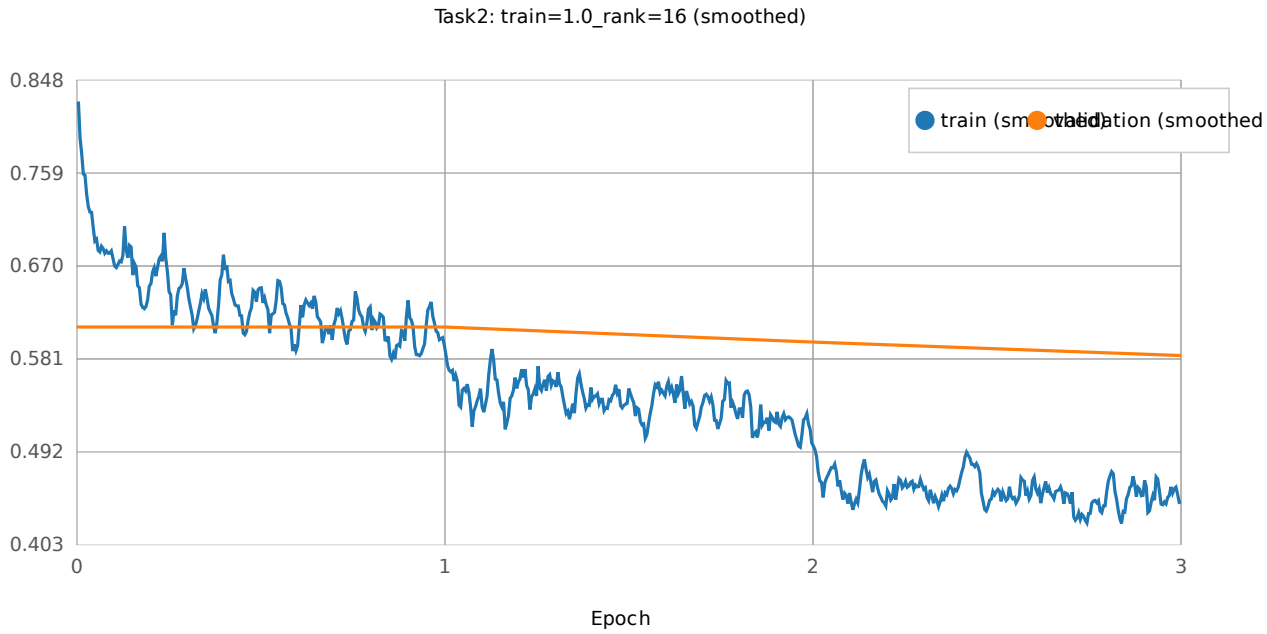
(PNG: `figures/task2_train-0p5_rank-8_train_val_smoothed.png`)

- train=1.0_rank=8:



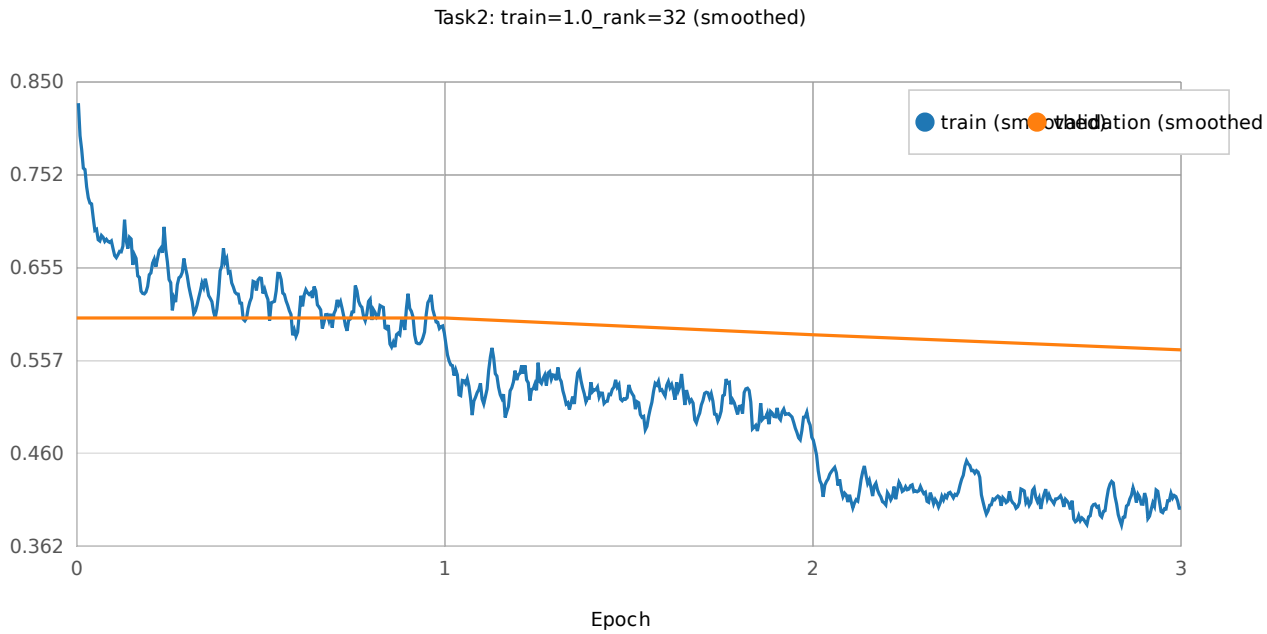
(PNG: figures/task2_train-1p0_rank-8_train_val_smoothed.png)

- train=1.0_rank=16:



(PNG: figures/task2_train-1p0_rank-16_train_val_smoothed.png)

- `train=1.0_rank=32`:



(PNG: `figures/task2_train-1p0_rank-32_train_val_smoothed.png`)

---

4. Results

Task 1 - RoBERTa QA

The table below lists the primary evaluation metrics (Exact, F1).

Configuration	Exact	F1
`pretrained` baseline	27.28	27.92
`rank=8_size=0.3`	61.43	70.32
`rank=8_size=0.5`	65.09	74.03
`rank=8_size=1.0`	68.07	77.08
`rank=16_size=1.0`	67.88	76.80
`rank=32_size=1.0`	68.08	77.08

Analysis: The lowest fine-tuned model showed a massive increase in performance over the baseline pretrained model (Exact: 27.28% -> 61.43% | F1: 27.92% -> 70.32%). Increasing the fraction of training data (at fixed LoRA rank=8) consistently improved both Exact Match and F1 sores: Exact increased from **61.43** (30% training data) → **65.09** (50% training data) → **68.07** (100% training data); F1 increased from **70.33** → **74.03** → **77.09** (roughly +6.6 Exact and +6.8 F1 from 30% to 100% training size). The effect of increasing LoRA rank (at full training size) was more modest: the highest-rank run (`rank=32_size=1.0`) achieves the highest Exact (68.09) and near-identical F1 (77.09) compared to `rank=8_size=1.0` (Exact 68.07, F1 77.09), while `rank=16_size=1.0` is slightly lower (Exact 67.89, F1 76.81). Overall, training size had a stronger and more consistent positive effect on performance than increasing rank beyond 8.

Task 2 - Decoder Generated Python Code

Model Configuration	BLEU Score	Execution Accuracy	Syntax Pass Rate	Execution Matches
Pretrained (Baseline)	9.41	3.18%	39.31%	79
rank=8_size=0.3	32.96	31.92%	54.43%	814
rank=8_size=0.5	33.96	33.47%	52.66%	833
rank=8_size=1.0	36.06	35.77%	52.26%	889
rank=16_size=1.0	37.08	37.06%	51.71%	922
rank=32_size=1.0 (Best)	38.55	38.29%	51.61%	952

Analysis: There was a massive improvement in execution accuracy from the baseline pretrained model to the lowest fine-tuned model (rank=8_size=0.3), Execution Accuracy: 3.18% -> 31.92%. Increasing the size of the training set with a constant rank 8 showed moderate gains: size 30% -> size 50%: (+1.5% accuracy), 50% -> 100%: (+2.3% accuracy). Decreased LoRA rank, on the other hand, showed very little decrease in accuracy: rank 32 -> rank 16 -> rank 8 (less than 1% drop in accuracy). BLEU score and execution accuracy showed steady gains from the lowest fine-tuned model (rank=8_size=0.3) to the heighest (rank=32_size=1.0), while syntax pass rate steadily dropped: 54.43% -> 38.29%. This suprising result is discussed in the Discussion section below.

5. Discussion

Overall, the Task 1 fine-tuned RoBERTa models performed as expected, with training size steadily improving performance, while LoRA rank only showed modest improvements. In the decoder models of Task 2, the surprising result of sytax pass rate consistently dropping with the higher-level fine-tuned models while the corresponding BLEU scores consistently improved may be explained by the higher-level models developing more sophisticated strategies for producing correct output (evidenced by their higher execution accuracy) with the trade-off of more frequently producing code that would not run (evidenced by the lower syntax pass score).

6. What I learned from all projects in this semester

- PA1 taught me to make an MLP "from scratch" using Torch. So, I learned quite a bit about using torch classes to make an FFNN, including choosing an optimizer and playing with various values for the width and depth of the NN.
- PA2 taught me how to work with masking and padding in an encoder-decoder transformer architecture as well as familiarity with seq2seq metrics like BLEU.
- PA3 taught me how to processes data to adapt it to specific pre-trained HuggingFace models (encoder-only and decoder-only models). It also taught me how to adjust parameters of HuggingFace trainers so that LLMs can fine-tune efficiently on a slurm-controlled GPU cluster with limited resources.
- All in all, I learned how to apply various pre-processing and evaluation strategies, gained familiarity with Torch, HuggingFace and Slurm, and I gained confidence in building custom architectures for custom tasks.